

Lenguajes Formales y De Programación

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas



Horarios Analizador léxico Para horarios Académicos

Manual Técnico — proyecto_1 (HorarioScript)

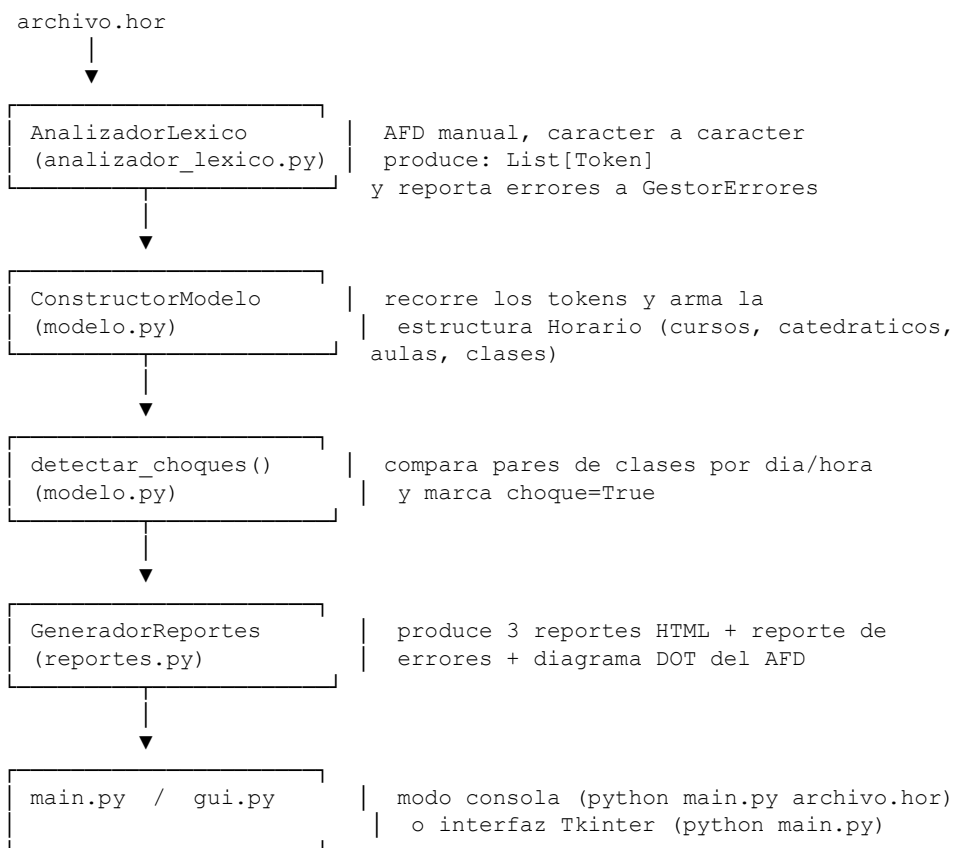
Analizador Léxico para Horarios Académicos

Lenguajes Formales y de Programación — Segundo Semestre 2026

Arquitectura del Sistema

El sistema se organiza en capas independientes, cada una con una única responsabilidad, comunicadas mediante objetos simples (`Token`,

`Horario`, listas y diccionarios):



`main.py` decide el modo de ejecución: si se le pasa un argumento

(`sys.argv[1]`), corre `ejecutar_consola()`; si no, importa `gui.py` y

abre la ventana.

Estructura del repositorio

```
proyecto_1/
├── main.py                      # Punto de entrada (consola o GUI)
├── .gitignore
├── clases/
│   ├── tokens.py               # Token, TipoToken, TipoError
│   ├── errores.py              # ErrorLexico, GestorErrores
│   ├── analizador_lexico.py    # AFD manual: siguiente_token()
│   ├── modelo.py               # ConstructorModelo, detectar_choques()
│   ├── reportes.py             # GeneradorReportes
│   └── gui.py                   # AplicacionProyecto1 (Tkinter)
├── ejemplos/
│   ├── horario_valido.hor
│   ├── horario_con_errores.hor
│   ├── horario_con_choque.hor
│   ├── nivel1_valido.hor
│   ├── nivel2_errores_lexicos.hor
│   └── nivel3_choque_de_horario.hor
└── reportes/                   # Salida generada en tiempo de ejecucion
    ├── reporte_horario_semanal.html
    ├── reporte_carga_catedraticos.html
    ├── reporte_estadistico_general.html
    ├── reporte_errores_lexicos.html
    └── afd_proyecto1.dot
```

Todos los módulos de `clases/` se importan de forma plana

(`from tokens import ...`, `from modelo import ...`), por lo que

`main.py` y `gui.py` agregan esa carpeta a `sys.path` antes de

importar:

```
```python
```

```
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "clases"))
```

```
3. Diagrama de Clases
```

Token

+ numero, lexema, tipo, linea, columna

+ to\_dict()

TipoToken (constantes)

RESERVADA\_BLOQUE, RESERVADA\_ELEMENTO, RESERVADA\_RELACION, CODIGO,

CADENA, HORA, ENTERO, DIA, CATEGORIA, IDENTIFICADOR, SIMBOLO,

COMENTARIO\_LINEA

TipoError (constantes)

CARACTER\_NO\_RECONOCIDO, CADENA\_SIN\_CERRAR, HORA\_FUERA\_DE\_RANGO,

DIA\_NO\_RECONOCIDO, CODIGO\_MAL\_FORMADO

ErrorLexico

+ numero, lexema, tipo\_error, descripcion, linea, columna

GestorErrores

- \_errores: List[ErrorLexico]
- + reportar(...) -> nunca detiene el analisis (modo panico)
- + hay\_errores(), total\_errores(), obtener\_errores(), limpiar()

AnalizadorLexico

- texto, pos, linea, columna, contador\_tokens
- gestor\_errores, \_ultimo\_identificador
- + siguiente\_token() -> Token | None
- \_reconocer\_comentario(), \_reconocer\_cadena()
- \_reconocer\_numero\_u\_hora(), \_reconocer\_hora()
- \_reconocer\_identificador(), \_clasificar\_identificador()

Curso / Catedratico / Aula / Clase

(entidades del dominio; Clase.choque marca conflictos)

Horario

+ cursos, catedraticos, aulas (dict por codigo), clases (list)

ConstructorModelo

- tokens, pos, n, horario
- + construir() -> Horario
- \_procesar\_elemento(), \_procesar\_clase(), \_leer\_bloque\_atributos()
- \_consumir\_simbolo(), \_consumir\_palabra(), \_consumir\_tipo()

detectar\_choques(horario) -> List[Tuple[Clase, Clase]]

(funcion de modulo, no metodo)

GeneradorReportes

- + generar\_reporte\_horario\_semanal()
- + generar\_reporte\_carga\_catedraticos()
- + generar\_reporte\_estadistico()
- + generar\_reporte\_errores()

+ generar\_dot\_afd()

+ generar\_todos()

AplicacionProyecto1(tk.Tk)

+ cargar\_archivo(), analizar(), abrir\_reporte()

## 4. Diagrama del AFD

El autómata completo se generó con Graphviz a partir del código DOT que produce ``GeneradorReportes.generar_dot_afd()``` (``reportes/afd_proyecto1.dot```), renderizado con:

```
```bash
dot -Tpng reportes/afd_proyecto1.dot -o reportes/afd_proyecto1.png
```

El autómata parte siempre de q_0 y se ramifica según el primer

carácter leído:

Desde q_0 con...	Va a...	Reconoce eventualmente
<code>`#`</code> seguido de otro <code>`#`</code>	$qC1 \rightarrow qC2$	<code>`COMENTARIO_LINEA`</code>
<code>`"``</code>	$qS1$	<code>`CADENA`</code> (o error <code>`CADENA_SIN_CERRAR`</code>)
dígito	$qN1$	<code>`ENTERO`</code> , o si sigue <code>`:`</code> $\rightarrow qH1 \rightarrow$ <code>`HORA`</code>
letra	$qA1$	palabra reservada / <code>`DIA`</code> / <code>`CATEGORIA`</code> / <code>`IDENTIFICADOR`</code> , o si sigue <code>`-`</code> $\rightarrow qCOD1 \rightarrow$ <code>`CODIGO`</code> (o error <code>`CODIGO_MAL_FORMADO`</code>)
<code>`{[]:;`</code>	—	<code>`SIMBOLO`</code>
espacio / tab / <code>`\n`</code>	q_0 (bucle)	no genera token
cualquier otro carácter	—	error <code>`CARACTER_NO_RECONOCIDO`</code>

Tabla de transiciones

Estado	Entrada	Estado destino	Acción	
q_0	<code>`#`</code>	$qC1$	esperar segundo <code>`#`</code>	
$qC1$	<code>`#`</code>	$qC2$	inicia acumulación de comentario	
$qC2$	<code>`char \n`</code>	$qC2$	acumula	
$qC2$	<code>`\n`</code> / EOF	ACEPTA	emite <code>`COMENTARIO_LINEA`</code>	
q_0	<code>`"``</code>	$qS1$	inicia cadena	
$qS1$	<code>`char " , \n`</code>	$qS1$	acumula	
$qS1$	<code>`"``</code>	ACEPTA	emite <code>`CADENA`</code>	
$qS1$	<code>`\n`</code> / EOF	ERROR	<code>`CADENA_SIN_CERRAR`</code>	
q_0	dígito	$qN1$	acumula parte entera	
$qN1$	dígito	$qN1$	acumula	
$qN1$	<code>`:`</code>	$qH1$	posible <code>`HORA`</code>	
$qN1$	otro	ACEPTA	emite <code>`ENTERO`</code>	
$qH1$	dígito	$qH1$	acumula minutos	
$qH1$	otro	ACEPTA	emite <code>`HORA`</code> (valida <code>`06:00`</code> – <code>`21:00`</code>)	
q_0	letra	$qA1$	acumula palabra	

qA1	letra\	dígito	qA1	acumula
qA1	`-`	qCOD1	posible `CODIGO`	
qA1	otro	ACEPTA	clasifica palabra	
qCOD1	dígito	qCOD1	acumula sufijo	
qCOD1	otro, ≥1 dígito leído	ACEPTA	emite `CODIGO`	
qCOD1	otro, 0 dígitos leídos	ERROR	`CODIGO_MAL_FORMADO`	
q0	símbolo `{ } [ ] : , ; `	—	ACEPTA `SIMBOLO`	
q0	espacio/tab/`\n`	q0	ignora (silencioso)	
q0	otro carácter	—	ERROR `CARACTER_NO_RECONOCIDO`	

Algoritmo de Tokenización

`AnalizadorLexico.siguiente_token()` implementa el AFD anterior de

forma imperativa, sin usar el módulo `re` ni funciones de alto nivel

de cadenas (`split`, `find`), solo indexación de caracteres

(`self.texto[self.pos]`):

1. `_saltar_espacios()` descarta espacios, tabs y saltos de línea, actualizando `linea/columna` en cada avance.
2. Se examina el carácter actual y se delega en el sub-método correspondiente (`_reconocer_comentario`, `_reconocer_cadena`, `_reconocer_numero_u_hora`, `_reconocer_identificador`).
3. Cada sub-método acumula caracteres en una lista y los une con `"".join(...)` al final, evitando slicing sobre el texto completo.
4. Si detecta una condición de error, se reporta al `GestorErrores`` (que nunca detiene el análisis) y, cuando aplica, se continúa llamando recursivamente a `siguiente_token()` — esto es el **modo pánico**.
5. El proceso se repite hasta `self.pos >= self.longitud`, momento en que `siguiente_token()` retorna `None`.

`tokenizar_texto()` invoca `siguiente_token()` en bucle hasta recibir

`None`, acumulando la lista completa de tokens.

Construcción del Modelo (`ConstructorModelo``)

No es parte del AFD léxico: es un recorrido simple de la lista de tokens ya generada (se filtran los `COMENTARIO_LINEA`). Reconoce patrones con un puntero de posición:

- ``curso` / `catedratico` / `aula`:`<palabra>: "NOMBRE" [attr: valor, ...]``
- ``clase`: `clase: "CURSO" con "DOCENTE" en "AULA" [dia:..., inicio:..., fin:..., seccion:...]`

`_leer_bloque_atributos()` lee genéricamente cualquier bloque `[clave: valor, ...]` y lo devuelve como diccionario, reutilizado por las 4 entidades. Es tolerante a errores: si un token no calza con lo esperado, se descarta y se continúa, para no interrumpir la generación de reportes aunque el archivo tenga errores léxicos.

Detección de Choques de Horario

`detectar_choques(horario)` compara **cada par** de clases (`a`, `b`) con `i < j`:

6. Si ``a.dia != b.dia`` (o alguno es ``None``), se descarta el par.
7. Se determina si comparten ``catedratico_codigo`` o ``aula_codigo``. Si no comparten ninguno, se descarta el par.

8. Se convierten ``inicio`/`fin`` a minutos desde medianoche.
9. Se aplica la prueba de traslape de intervalos:

`ini_a < fin_b and ini_b < fin_a`. Si se cumple, ambas clases se marcan `choque = True` y el par se agrega a la lista de conflictos, que el Reporte 1 resalta en rojo.

Complejidad $O(n^2)$ sobre el número de clases — aceptable para el tamaño de horario de una facultad en un archivo `.hor`.

Generación de Reportes

`GeneradorReportes` recibe el `Horario` ya construido (con los choques marcados), la lista de tokens y la lista de errores, y produce 5 archivos en la carpeta `reportes/`:

Archivo	Contenido
---------	-----------

`reporte_horario_semanal.html`	Una tabla por sección, ordenada por día/hora. Filas verdes (`CONFIRMADO`) o rojas (`CHOQUE DE HORARIO`) según `clase.choque`.
`reporte_carga_catedraticos.html`	Por cada catedrático: categoría, horas semanales (suma de duración de sus clases) y número de clases asignadas, con nivel de carga (BAJA/NORMAL/ALTA/SATURADA) resaltado por color.
`reporte_estadistico_general.html`	Indicadores generales del ciclo (totales, catedrático con mayor carga, promedio de horas) y tabla de ocupación por aula, resaltando en rojo las que superan 80%.
`reporte_errores_lexicos.html`	Tabla de errores léxicos: número, lexema, tipo, descripción, línea y columna.
`afd_proyecto1.dot`	Código DOT con el diagrama completo del AFD (ver sección 4).

El CSS es mínimo (~15 líneas) y se comparte entre los 4 reportes HTML

mediante `_envoltorio_html()`.

Umbral de carga académica (`\_nivel\_carga`)

Horas semanales	Nivel	Color
1 – 4	BAJA	azul
5 – 10	NORMAL	verde
11 – 15	ALTA	naranja
16+	SATURADA	rojo

Interfaz Gráfica (`gui.py`)

`AplicacionProyecto1` (Tkinter) tiene:

- Una barra superior con 6 botones: **Cargar .hor**, **Analizar**, y 4 accesos directos a los reportes (**Reporte 1/2/3, Errores**), deshabilitados hasta que se ejecuta un análisis.
- 3 pestañas: **Archivo** (vista previa del `.hor` cargado), **Tokens y Errores** (tablas `Treeview`).
- Una barra de estado inferior con el resumen del último análisis
(N tokens | N errores | N clases | N choques).

El botón `Analizar` ejecuta el pipeline completo (tokenizar → construir modelo → detectar choques → generar reportes) y habilita los botones de reporte, que abren cada HTML en el navegador predeterminado (`webbrowser.open()`).

Justificación de Decisiones de Diseño

- **Acumulación de lexemas en listas + `"".join`**: evita la

concatenación repetida de cadenas inmutables (`str += char`, costo cuadrático en el peor caso) y evita usar slicing sobre el texto completo, respetando la restricción de "solo indexación".

- **`detectar_choques`` como función de módulo, no método:** se separa de `ConstructorModelo` porque conceptualmente opera sobre un `Horario` ya construido, no sobre el recorrido de tokens; mantiene cada función con una única responsabilidad.
- **`_leer_bloque_atributos`` genérico:** las 4 entidades del lenguaje (`curso`, `catedratico`, `aula`, `clase`) comparten la misma sintaxis de atributos [`clave: valor, ...`], así que un solo método cubre las cuatro sin duplicar lógica de parseo.
- **Reportes tolerantes a datos incompletos:** si un archivo `.hor`` tiene errores léxicos, `ConstructorModelo` igual construye el mejor modelo posible con lo que logró reconocer, y `GeneradorReportes` genera los 5 archivos de todas formas — el reporte de errores documenta lo que falló, pero no bloquea el resto del análisis.
- **`main.py`` con doble modo (consola / GUI):** facilita probar el motor léxico de forma rápida y scriptable (`python main.py archivo.hor``) sin necesidad de abrir la interfaz gráfica, útil para correr los casos de prueba de forma repetible.

Diagrama AFD

